Chapter 12
Model Operationalization at Edge Devices

Shikhar Kwatra, Utpal Mangla, and Mudhakar Srivatsa

Abstract This Chapter covers the core aspects related to Model Operationalization (ML-Ops). *MLOps* accelerates the journey of seamlessly building, training, validating and deploying the optimal models in production. This allows for the organization to accelerate their end to end pipeline from capturing the data from different sources and applying governance policies on the data to continuously monitoring the best candidate models deployed in production environment. This chapter further articulates the intersection of ML-Ops at edge devices and optimizations associated with efficiently creating ML-Ops pipeline across said edge devices.

1 Model Operationalization at a Glance

Data Engineers, Data scientists or Business Analysts, at scale have been experimenting with a variety of deep learning and machine learning models. Furthermore, in most enterprises, these personas have been involved in different stages of model creation, be it at model validation stage or at the step of presenting detailed insights to the third parties or respective customers.

According to a research conducted in 2016 (https://visit.figure-eight.com/rs/416-ZBE-142/images/CrowdFlower_DataScienceReport_2016.pdf), it is stated that 80% of the models created by other data scientists are side projects that are not put into production. Furthermore, there is widespread consensus among ML practitioners that data preparation accounts for approximately 80% of the time spent in developing a viable ML model (https://www.forbes.com/sites/gilpress/2016/03/23/data-preparation-most-time-consuming-least-enjoyable-data-science-task-survey-says/?sh=2fb540636f63).

S. Kwatra (✉)
AWS, Santa Clara, CA, USA
e-mail: sk4094@columbia.edu; shkwatra@amazon.com

U. Mangla · M. Srivatsa
IBM Thomas J. Watson Research Center, New York, NY, USA
e-mail: utpal@ibm.com; mudhakar@ibm.com

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

M. Srivatsa et al. (eds.), *Artificial Intelligence for Edge Computing*,
https://doi.org/10.1007/978-3-031-40787-1_12

If we examine the statistics pertaining to different models being used in various industries, such as the financial, retail, and healthcare sectors, we find that the majority of them are rule-based models.

ML operationalization is the process of putting machine learning models into use in the real world so that businesses may benefit from them. The DevOps paradigm is layered on the model lifecycle management process through ML operationalization (CRISP-DM). One aspect for this process is monitoring not only prediction quality, but also system resources. This is important to correctly provide the necessary infrastructure, either using a fully-managed cloud platform or a local solution [1].

ML models can be compared to the engine of a car from an analogies standpoint. But when you examine a car, you don't just look at the engine; you also consider the chassis, sideview mirrors, transmission, entertainment system, etc., which round out the overall spec and infrastructure of the vehicle. Similar to this, the idea of operationalizing machine learning models includes the use of ML models along with independent validation, deployment, supporting a scalable servicing infrastructure, and having a Continuous Integration/Continuous Deployment (CI/CD) pipeline.

2 Pain Points

Operationalization of machine learning or deep learning models is difficult due to the following pain points:

- Time taken from model *conceptualization* to *productionization*—It can take weeks or even months to push or *productionize* a particular model post conceptualization due to several components involved in creating a ML-Ops pipeline.
- Serving millions of requests for creating a scalable infrastructure
- Effort in pushing from black-box to glass-box approach leading to more transparency
- The ability to explain to auditors why a particular model is predicting in a certain way
- Need to get Explanation for every case predicted by a specific model
- Need to have an optimized Infrastructure to support large number of Data Scientists and respective models

3 ML-Ops Key Metrics

In order to solve the above problems, Operationalization of models either at the cloud or edge needs to be solved, taking into consideration the following key metrics:

- Continuous Training

- Automated Validation and Deployment
- Insight Infusion at Scale
- Ensuring Transparency
- Removing Bias
- Business KPI Mapping
- Data and Model Governance
- Model Risk Management

4 Features of Model Operationalizaition Framework

In terms of inspecting the MLOps frameworks, the following features need to be considered for successful environment setup:

Features	Description
Flexibility or Customizability	How flexible is the platform in integrating and/or customizing new frameworks for AI model development.
Ease of Use	How easy is it to leverage these tools and proposed techniques from setup to application.
Integrations	How well does the platform integrate with Git or other model versioning and source control tools, catalogs (for governance and discoverability) or various data sources.
Governance	How well does the solution support governance and discoverability of assets (data assets, models, notebooks etc.)
Platform	Support for various platforms (public cloud, on-prem, hybrid cloud), and compute types (CPU/GPU) for training and scoring (or inference) AI models
Monitoring	How well does the solution support monitoring AI models for performance, explainability, fairness etc.
Scalability	How scalable is the platform in supporting various Data and AI users in different roles to explore, develop, and deploy AI models.
Openness	How well does the platform support open-source technologies which has become a key differentiator for platform providers.
Security	How well does the platform support enterprise-grade security access to the platform in terms of authorization and authentication.
Support for 5Cs	Support for continuous training, validation, deployment, integration and monitoring

5 Popular ML-Ops Frameworks

There are various frameworks and platforms available for ML Operationalization with varied degree of support for required Functional and Non Functional capabilities as demonstrated in the image below (Fig. 12.1).

6 Key Capabilities Needed Across the Steps of ML-Ops Pipeline

The flow shown below depicts different steps across the workflow that indicate the capabilities needed to build a successful ML-Ops life-cycle. The personas involved across each stage, for instance, Data Steward, Data Engineer may change based on the users within a respective organization who are involved in the end-to-end life-cycle of different machine learning projects.

In contrast to Application DevOps, ML-Ops not only takes care of Continuous Integration and Continuous Deployment but also covers Continuous Training, Continuous Validation and Continuous Monitoring (Fig. 12.2). [3] covers the various tooling that have emerged in automating MLOps pipeline.

6.1 Data Provisioning with Governance

This primary stage involves data Ingestion from various types of Data Sources – File Servers, Object Stores, Relational Databases, NoSQL, Hadoop, Rest Based Data

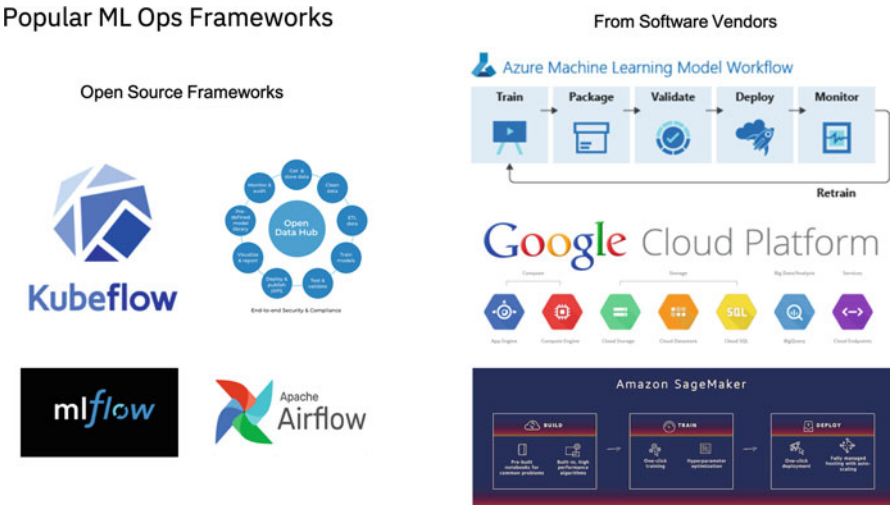

Fig. 12.1 Enterprise ML-Ops frameworks

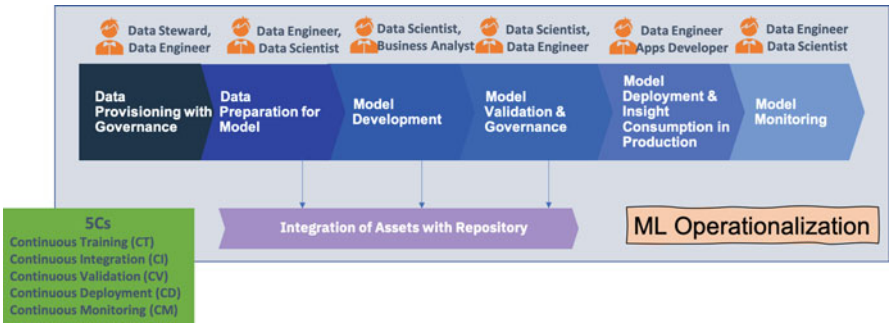

Fig. 12.2 ML-Ops pipeline

Services etc. This also implies metadata discovery, forming the data lineage, data quality control, overlaying canonical models, and enforcing of data security and data policies, along with virtualization of data including caching and joining between varied relational and non-relational data stores.

Data Lineage within the data provisioning layer aids in understanding, recording, and visualizing the data as it flows from a plethora of data sources to consumption. This can also include several data transformations that were executed and captured on the data itself. Data lineage can be further corroborated with tags for marking the data appropriately across the transformation lifecycle. Another form of data lineage may include parsing which relies on automatically reading logic such as SQL-based solutioning, extract transform load operations, etc. in order to process the data.

Akin to Data Lineage, Model Lineage assists in tracking data scientists and model builders with :

- Keeping a running history of model discovery experiments.
- Establishing model governance by tracking model lineage artifacts for auditing and compliance verification.

In medium-to-large scale enterprises, data lineage is infused with data classification as part of data governance as it plays an important part of an information security and compliance program. It can help in understanding and encapsulating sensitive data or critical information as data is parsed across a cross collaboration matrix of data engineers, data stewards, data scientists and data admin.

Hence, when it comes to data governance and classification, data protection plays a vital role. It involves many layers of protection including data loss prevention that aids in inspecting data in motion, or at rest, regardless of its location at a cloud or edge device. With data masking and encryption, encapsulating or obfuscating sensitive data like Social Security information, Date of birth or other PII (Personal Identifiable information) including health records can be prevented from being shared across multiple unauthenticated parties or get stolen.

Similarly, additional layers of protection Identity access management which can pertain to monitoring data access and activities of privileged users, database

firewall for blocking Sql injection and common vulnerabilities, and alerting module to proactively and reactively alert the user of any suspicious activity with the data.

Data lineage assists users in ensuring their data is coming from a trusted source, has been transformed correctly, and loaded to the specified location. Hence, data becomes very difficult to verify or very costly and time-consuming, if such data processes are not tracked correctly. Even in Data Migrations, having a clearly defined data lineage can provide the tracking of information across multiple sources and access to such information quickly and easily.

6.2 Data Preparation for Models

As part of data selection, the user should be able to query the data from a variety of data sources, either via importing JDBC/ODBC driver or connecting via credentials secured by the authorized persona. Data scientists may not need access to all the data sources or data connections, and hence, data steward or data engineer may provision access to certain relevant data sources pertaining to the use case under consideration.

This stage of data preparation dives deeper into data selection, cleansing, exploration, and visualization of data. Feature engineering or data transformation can involve impute missing data with mean or medium, one hot encoding, masking sensitive or confidential information like names or addresses and time-series specific transformers to accelerate the preparation of time series data for ML.

Such feature engineering can be done either using UI based tools provided by different organizations like Data Refinery (IBM), Amazon Sagemaker Data Wrangler (AWS) etc. and via support for various Programming Paradigms inclusive of but not limited to Python, R, Scala, SQL etc. Simultaneously, support for industry standard IDEs including Rstudio, Jupyterlab etc. need to be handled with data profiling and visualization capabilities.

This should also involve handling large volumes of data via distributed computing and pushing different processes to data lake or data stores wherever needed. Data Preparation step should identify potential errors and extreme values with a set of robust pre-configured visualization templates including histograms, scatter plots, box and whisker plots, line plots, and bar charts etc. There are statistical techniques to further detect potential bias during data preparation.

Once your data is prepared, fully automated ML workflows can be built and pushed to the model building and development lifecycle.

6.3 Model Development

This stage involves building different machine learning models post data transformation. Further feature engineering may be done by the data scientist prior

to building different machine learning models using various Machine Learning frameworks and libraries including TensorFlow, Keras, PyTorch, Scikit-learn, Spark etc.

For instance, optimization problems where linear programming and genetic algorithms are involved can be solved via Decision Optimization tools. Data Scientists or ML engineers use various industry standard IDEs including Rstudio, Jupyter Notebooks, Jupyter Labs etc. for the purposes of automated model development by applying candidate Algorithms, Hyper Parameter Optimization, Mathematical Feature Engineering, Code Generation and API support.

Best practices as part of Model Development include Experiment tracking and consistent model versioning, which helps to keep track of changes and/or revert back to previous working versions in cases where the model starts under-performing to incoming data.

It is crucial to track the code for the algorithm used, hyperparameters used, any changes to the dataset and the kind of results obtained for a certain set of hyperparameters. Model development is essentially an iterative process where multiple models are tested until the model fits the desired criteria suitable for staging the model or pushing the optimal model in production.

Various UI based automated machine learning tools exist inclusive of but not limited to AutoML, Sagemaker Autopilot, IBM AutoAI that eliminate the heavy lifting of building various candidate machine learning models and tuning the best ML Models based on the data, while maintaining full control and visibility, either via UI or API based access to the data scientists authorized to work on the use case. Some of these tools including Autopilot and AutoAI can generate model candidate notebooks with Model Explainability, showcasing how the model pipelines created feature engineering and model development steps. It also provides a feature importance score to showcase the relative importance of each input feature affecting the target label. Providing these notebooks can aid data scientists to accelerate the model building journey by directly performing custom operations in the generated notebooks and modifying the same as part of model development iterations.

Hence, we can constantly track the performance requirements of the models and deploy the model to production or iterate on the recommended solutions to further improve the model quality, thereby substantially reducing the likelihood of model drift as these optimal models are pushed into the production environment to handle live unseen traffic.

6.4 Model Validation and Governance

Independent model validation becomes an integral component of model risk management. Open Risk Manual defines Model Governance as:

An internal framework of a firm or organization that controls the processes for model development, validation and usage, assigns responsibilities and roles, etc.

Model Governance framework is needed to understand:

- Ownership of one or multiple models by the authorized persona
- Model compliance with relevant regulations and guidelines
- Model Artifacts information including training and validation data
- Performance tracking thresholds or approvals required at various stages of model development
- Post-deployment model monitoring means

Hence, this stage in the pipeline involves validating the developed machine learning and deep learning models with blind test data for Accuracy, Fairness, Potential Drift (data and model drift) and local feature importance.

Model validation should occur with proper model artifact versioning and model governance through workflow, while simultaneously, maintaining a proper model lineage. ML-Ops engineers/Model Validators are able to move approved model to production stage for continuous monitoring.

Model factsheets can also be created to showcase performance insights and model metadata. A factsheet is considered a collection of relevant information (facts) about the creation and deployment of an AI model or service. Facts could range from information about the purpose and criticality of the model, measured characteristics of the dataset, model, or service, or actions taken during the creation and deployment process of the model or service.

These model factsheets developed by IBM are tailored to the particular AI model or service being documented, and thus can vary in content. They are tailored to the needs of their target audience or consumer, and thus can vary in content and format, even for the same model or service. They capture model or service facts from the entire AI lifecycle; and said factsheets are compiled with inputs from multiple roles in this lifecycle as they perform their actions to increase the accuracy of these facts.

6.5 Model Deployment and Insight Consumption in Production

ML-Ops engineers or respective similar ML Model deployer personas in the industry are responsible for packaging and deploying the machine learning and deep learning models using various UI and API-based deployment for models.

Large enterprises working on hundreds of models at scale, with cross-collaboration of data scientists, data stewards, administrators, and business analysts are pushing dozens of optimal models in production pertaining to a use case under consideration. The purpose of deploying your model ensures you can make the predictions from a trained ML model available to others, whether that be users, management, or other systems.

There are several methods to deploy ML Models:

- **Batch or Offline Deployment**—Allows to take batches or sub-samples of data at a particular time, and get the batch predictions in offline or in other words,

not requiring in real-time. Support for batch and online processing needs to be in place depending on the use case, along with support for high throughput Scoring requests from Consuming Applications with Scalability, Elasticity and Failover.
- **Real-Time Deployment**—There are certain cases where the users may want the prediction in real-time, for instance, determining the likelihood of the customer to churn or if the transaction is fraudulent or not. In such a case, there needs to be support for the REST interface which can be invoked via the endpoint to access the model for getting the prediction.

6.6 Model Monitoring

This stage supports continuous monitoring of models for online and offline inference with key performance metrics including accuracy, drift, fairness (to ensure trustworthiness in the models) along with Model Explainability to get reasoning behind the prediction as to why the model predicted a certain output.

Model monitoring can include:

- Data Quality monitoring: Monitoring the data draft. As more data or real time traffic is incoming to invoke the model endpoint, there might be drift in feature values that can degrade or modify the performance metrics of the model.
- Model Quality Monitoring: Model performance metrics, for instance, F1 Score or accuracy can be affected due to drift in model quality.
- Bias of Models in production—Due to imbalance in the data or skewed data for a use case, model's predictions may result in biased outcomes.

Tools like IBM AutoAI or Amazon Sagemaker can assist in determining Bias, model quality or data drift to create a fair model version. Logs can be captured to further send notifications to the authorized persona indicating the drift in model, data or bias being highlighted in the results.

Adding Human in the loop or having an aggregate can provide measures to ameliorate the results or model performance through balancing the data, generating synthetic data or taking care of bias in the data by taking into account fairness measures accordingly.

6.7 Integration with Source Code Repository

Adding automation to your workflow doesn't end with deploying code into production. Users have to keep track of new errors before they significantly impact them.

This stage involves efficient integration of model and data artifacts along with key metadata information to be tracked and stored in respective data and model repositories, for instance, GitHub, GitLab, BitBucket or other private container repositories.

The ML-Ops engineers should be able to leverage a UI based approach using existing Integration tools or API based approach.

6.8 Continuous Training, Integration, Validation Deployment and Monitoring

This stage is a universally running stage across the entire ML-Ops pipeline, wherein the ML-Ops engineer or persona responsible for pipeline workflow, should perform integration with external workflow tools. Said external tools include Jenkins, Proprietary Tools using Scripts and/or Command Line Interface (CLI) or proprietary internal workflow tools in order to maintain the 5Cs as part of best practices for operationalization of machine learning models. In [2], the team built a functional ML platform with DevOps capability from existing continuous integration (CI) or continuous delivery (CD) tools and Kubeflow, constructed and ran ML pipelines to train models with different layers and hyperparameters while time and computing resources consumed were recorded. This provides a valuable reference for ML pipeline platform construction in practice.

GitLab has Continuous Integration/Continuous Delivery (CI/CD) and DevOps workflows built-in. GitHub lets you work with the CI/CD tools of your choice, will need to be manually integrated by the users. GitHub users typically work with such third-party CI programs such as Jenkins, CircleCI, or TravisCI.

7 ML-Ops Intersection with Edge Computing

As articulated in previous sections, currently machine learning models are trained in cloud on servers with high compute and storage capacity. In order to get the inference from these models in the production environment, live data was processed and sent to the cloud via the internet gateway, invoke the rest endpoint (in case that model is packaged and deployed in production and can be invoked with REST API), get the results back from the cloud to the edge device and then perform the respective output as per the inference.

The aforementioned sequence for ML-Ops is applicable to edge computing as we are able to push models to the edge platforms and devices with efficient packaging of models and optimizing the sequence of operations performed to make them compatible to be executed on such edge devices.

Model frameworks such as Tensorflow, MXNet, PyTorch, XGBoost etc. can be optimized to be easily deployed across a fleet of devices independent of firmware and application updates. Automated ML-Ops pipelines can be created for edge servers, smart cameras and Internet of Things (IoT) enabled sensors.

For instance, a computer vision emotion classifier model with the ability to recognize the emotions of the user based on user's expressions, first transfers the user's features at a given point of time to be classified by the emotion classifier to the edge device, further processed and inference regarding the emotion is delivered with near real time latency. In order to eliminate the latency, the machine learning models can now be optimized and deployed on the edge device without the need to process the data in the cloud. This leads to the rise of Operationalization of complex models at edge devices using the power of edge computing, IoT and ML-Ops.

As witnessed in the pain points, in ML-Ops in larger environments, from a general perspective, there are different sets of challenges incurred with respect to model deployments when targeted towards IoT enabled/edge devices.

- Since the models need to be deployed on different types of target platforms, there may be a lack of standardization or in other words, the models need to be highly device centric and hence, customized for each target platform along with taking security constraints, environment-based packaging and performance metrics into consideration.
- Such IoT based machine learning models need to be continually trained in a frequent manner as they tend to degrade faster due to rapidly drifting IoT data being ingested by such models.
- Inferences should be able to in online as well as offline mode since some IoT enabled Edge solutions may require offline operations with rapidly refreshing the model versions to adapt to such IoT solutions.

As different models are trained in the cloud, they need to be edge-ready. Hence, the models need to transformed to low-level framework agnostic operational understanding.

8 Edge Devices with MLOps Pipeline

Tools and frameworks like Amazon Sagemaker Edge can be used to operate ML Models on a fleet of devices. In a particular MLOps compute on edge scenario, we set an edge agent deployed on the edge device to sync with the edge manager. Edge agent is configured to capture the data from said edge devices to the cloud. We programmatically set triggers and sampling frequency to upload sampled data securely from any device. The agent runs on the CPU of your devices.

The agent runs inference on the framework and hardware of the target device you specified during the compilation job. For example, if you compiled your model for the Jetson Nano, the agent supports the GPU in the provided Deep Learning Runtime (DLR).

Once the edge agent has captured the sampled data, it stores the data to a data lake, such as cloud repository bucket like S3 for analysis and visualization.

Then, model can be built and trained using UI or custom modeling notebooks running in the cloud. The updated model satisfying the performance criteria goes

through the optimization stage to create an optimized model version leveraging available hardware for edge inference. The optimized model is then packaged and signed to be device compatible. The model package if further deployed to fleet of edge devices. Monitoring agent can be deployed on the fleet of edge devices to continuously monitor all the models. Once the monitoring of models on edge devices is turned on, multiple models can seamlessly run on the edge devices.

9 Model Deployment on Edge Devices

In collaboration with Intel, Amazon developed DeepLens—a wireless camera with AI inferencing capabilities. This allows ML models to be trained on SageMaker with support for different ML/DL frameworks and then to be deployed on DeepLens in an integrated fashion. Optimizing models to deploy on the edge compresses the model size so it runs quickly.

Google has also entered the field with the underlying hardware—Google's Edge Tensor Processing Units (TPUs). Coral provides the full toolkit to train TensorFlow models and deploy them on different platforms using the Coral USB Accelerator.

As another example, IBM Edge Application Manager (IEAM) deploys machine learning (ML) models. In an edge solution, there could be many models created and deployed. These models need to be managed, and that's where the model management system (MMS) comes in to play.

IBM leverages MMS can be used to deploy, manage, and synchronize models across the edge tiers. It will facilitate the storage, delivery, and security of models/data and other metadata packages needed by edge and cloud services. edge nodes can send and receive models and metadata to and from the hybrid cloud.

For deployment on such edge devices, it is important to convert the framework-specific functions and operations into a framework-agnostic intermediate representation. Then, it is required to perform a series of optimizations for which you need to know the hardware architecture, instruction set, memory access patterns, and input data shapes etc.

Then, we generate binary code for the optimized operations, write them to a shared object library, and save the model definition and parameters into separate files. Then, the compiled model can be loaded and executed on the target edge device.

Hence, several methods including designing power-efficient ML algorithms, developing better and more specialized hardware, and inventing new distributed-learning algorithms where all IoT devices communicate and share data can be leveraged to use said complex ML models at edge devices. The last approach is limited by the network bandwidth, therefore future 5G networks, which provide ultra-reliable, low-latency communication services, will help immensely in the area of edge computing.

10 Conclusion

We covered the core aspects of Model Operationalization (ML-Ops) and its popular frameworks. We walked through the various components involved within the ML-Ops pipeline and elucidated the best practices to optimize the journey of machine learning and deep learning models. Further, we discussed the intersection of ML-Ops at edge devices and optimizations associated with efficiently creating ML-Ops pipeline across edge devices.

References

1. L. Cardoso Silva et al., Benchmarking machine learning solutions in production, in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)* (2020)
2. Y. Zhou, Y. Yu, B. Ding, Towards MLOps: a case study of ml pipeline platform, in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)* (2020), pp. 494-500. https://doi.org/10.1109/ICAICE51518.2020.00102
3. P. Ruf, et al., Demystifying MLOps and presenting a recipe for the selection of open-source tools. Appl. Sci. **11**(19), 8861 (2021). https://doi.org/10.3390/app11198861